

## Plan of Action - DG Cities Project Reference

### Overview:

This document outlines the end-to-end architecture, container communication, data formats, permissions, workflow dependencies, and next steps for implementing Option A (batch predictions) within a three-container setup.

### 1. Containers:

#### a. preprocess (one-shot):

- Base: python:3.12-slim
- Runs on demand/manual or via GitHub Actions
- Mounts:
  - \* ./data/raw -> /data/raw (read)
  - \* warehouse volume -> /data/warehouse (write)
- Uses Polars + PyArrow to read CSVs and write Parquet: repairs.parquet, properties.parquet

#### b. backend (always-on API):

- Base: tiangolo/uvicorn-gunicorn-fastapi:python3.12
- Runs persistently on port 8000
- Mounts:
  - \* warehouse volume -> /data/warehouse (read-only)
- Uses DuckDB/Polars to query Parquet files
- Endpoints:
  - \* GET /repairs?property\_id=&year?&type?
  - \* GET /predictions?property\_id=

#### c. ui (static React):

- Base: nginx:alpine serving build/
- Runs on port 80
- Fetches only from backend:8000

### 2. Data Formats:

- Raw CSVs: source data, updated annually
- Parquet tables in /data/warehouse:
  - \* repairs.parquet: property\_id:int, date:date, repair\_type:str, cost:float, ...
  - \* properties.parquet: static metadata per property
  - \* predictions.parquet: property\_id:int, repair\_type:str, prob\_next\_12m:float, predicted\_date:date

### 3. Permissions & Access:

- preprocess: read/write to raw and warehouse dirs
- backend: read-only access to warehouse volume
- ui: no file access, only HTTP fetches

### 4. Container Communication:

- Data preparation -> Parquet via preprocess
- ML team writes predictions.parquet into warehouse
- backend queries Parquet on every HTTP request
- ui sends GET requests with query parameters, receives JSON

#### 5. Workflow Dependencies:

1. CSV update triggers preprocess (manual/GHA/cron)
2. preprocess generates Parquet tables
3. ML batch job consumes Parquet and outputs predictions.parquet
4. backend + ui start, reading fresh Parquet data

#### 6. Next Steps:

- Finalize preprocess.py and Dockerfile; test Parquet output
- Share Parquet schema & path with ML team; obtain predictions.parquet
- Implement main.py in backend with two GET endpoints; Dockerize & test
- Update React fetch calls to include filters; rebuild UI container
- Create docker-compose.yml for all services; verify end-to-end

Keep this plan as a reference for team coordination and Monday discussions.